Web API: архитектура, маршрутизация и обработка HTTP-запросов

МДК.02.01 Технология разработки программного обеспечения

Что такое API и зачем он нужен

API (Application Programming Interface) — это набор правил и способов, с помощью которых одна программа может взаимодействовать с другой.

API определяет:

- какие операции доступны;
- какие данные можно передавать;
- в каком формате отправлять запрос;
- какой результат можно получить;
- какие ошибки могут возникнуть.

Основные задачи API

- обмен данными между приложениями;
- доступ к серверной логике;
- работа с общей базой данных;
- интеграция разных систем;
- разделение клиентской и серверной частей.

API — это контракт между программами.

Виды API

- Library API — взаимодействие через классы, методы и функции библиотеки;
- Operating System API — доступ к возможностям операционной системы;
- Web API — взаимодействие между приложениями по сети;
- Hardware API — доступ программ к устройствам и оборудованию.

Подходы и технологии Web API

- REST — архитектурный стиль, ориентированный на ресурсы;
- SOAP — протокол обмена структурированными XML-сообщениями;
- RPC — модель удалённого вызова процедур или методов;
- gRPC — современный высокопроизводительный RPC-фреймворк;
- GraphQL — язык запросов и модель выполнения API.

Web API — общее понятие. REST, SOAP, RPC, gRPC и GraphQL относятся к разным уровням, но их удобно сопоставить как варианты взаимодействия.

REST, SOAP, RPC, gRPC и GraphQL: сравнение

Подход / технология	Основная идея	Частый формат	Типичное применение
REST	Работа с ресурсами	Обычно JSON	Web и мобильные приложения
SOAP	Обмен сообщениями по строгому контракту	XML	Корпоративные интеграции
RPC	Вызов удалённых функций	Зависит от реализации	Внутренние сервисы
gRPC	Быстрый удалённый вызов методов	Protobuf по умолчанию	Микросервисы
GraphQL	Клиент выбирает необходимые поля	Обычно JSON	Сложные клиентские приложения

REST API

REST (Representational State Transfer) — архитектурный стиль построения Web API.

ресурс имеет собственный URL;

действие определяется HTTP-методом;

запросы не должны зависеть от состояния предыдущих запросов;

для результата используются стандартные HTTP-коды;

данные часто передаются в JSON.

Пример ресурса Products

```
GET    /api/products
GET    /api/products/5
POST   /api/products
PUT    /api/products/5
PATCH  /api/products/5
DELETE /api/products/5
```

В этой теме дальше используется REST-подход в ASP.NET Core.

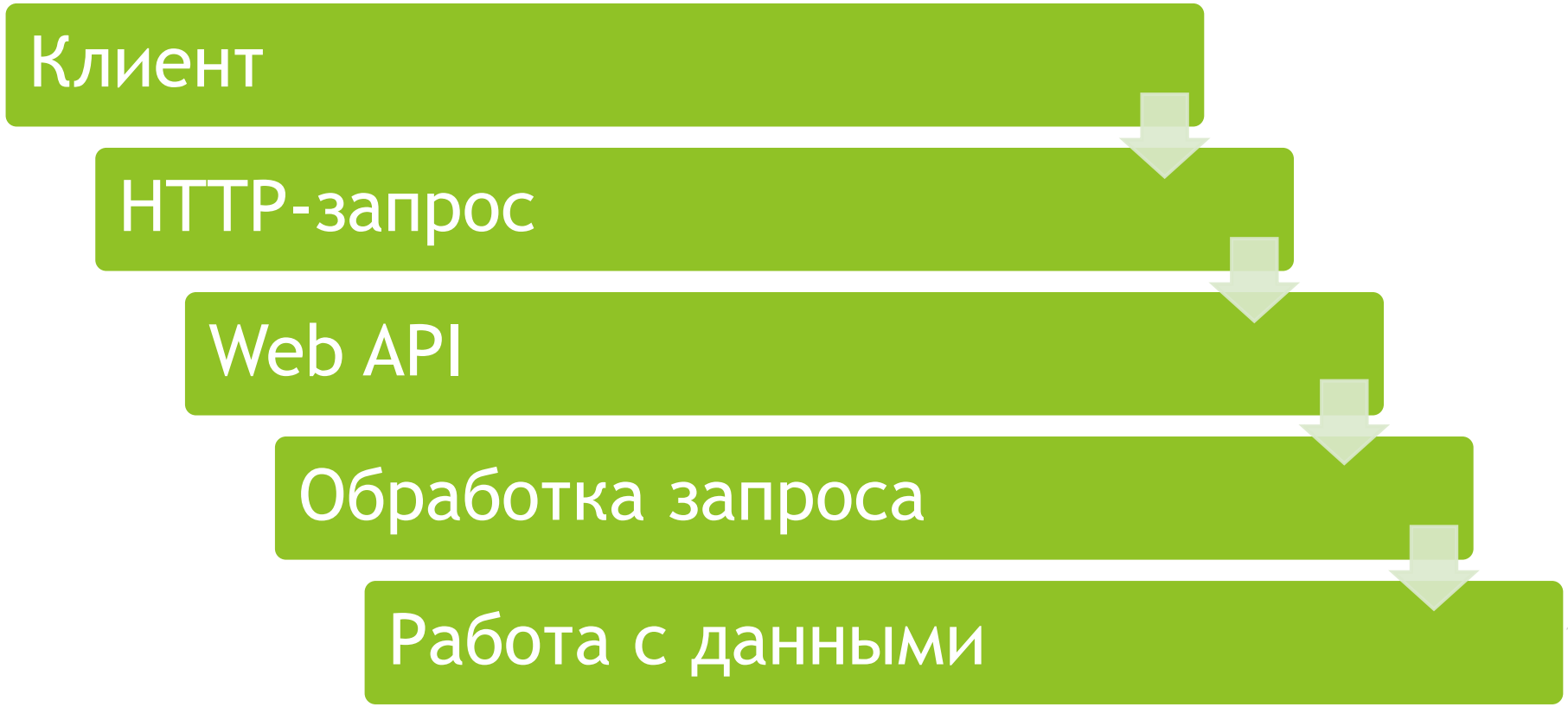
Web API

Web API обычно работает по сети и использует стандартные веб-технологии:

- HTTP / HTTPS
- URL
- HTTP-методы
- Заголовки
- JSON
- Коды состояния

Web API не привязан к конкретному интерфейсу пользователя. Один API может использоваться множеством разных клиентов.

Как работает Web API



Пример HTTP-запроса

```
POST /api/v1/products
Content-Type: application/json

{
  "name": "Keyboard",
  "price": 5000
}
```

Пример HTTP-ответа

```
HTTP/1.1 201 Created
Content-Type: application/json
{
  "id": 15,
  "name": "Keyboard",
  "price": 5000
}
```

Важный принцип Web API

Web API предоставляет данные и операции, а пользовательский интерфейс формируется на стороне клиента.

Основные HTTP-методы

Метод	Назначение	Пример
`GET`	Получить данные	Получить список товаров
`POST`	Создать новый ресурс	Добавить товар
`PUT`	Полностью изменить ресурс	Обновить данные товара
`PATCH`	Частично изменить ресурс	Изменить только цену
`DELETE`	Удалить ресурс	Удалить товар

Примеры запросов

Примеры запросов:

GET /api/products

GET /api/products/5

POST /api/products

PUT /api/products/5

PATCH /api/products/5

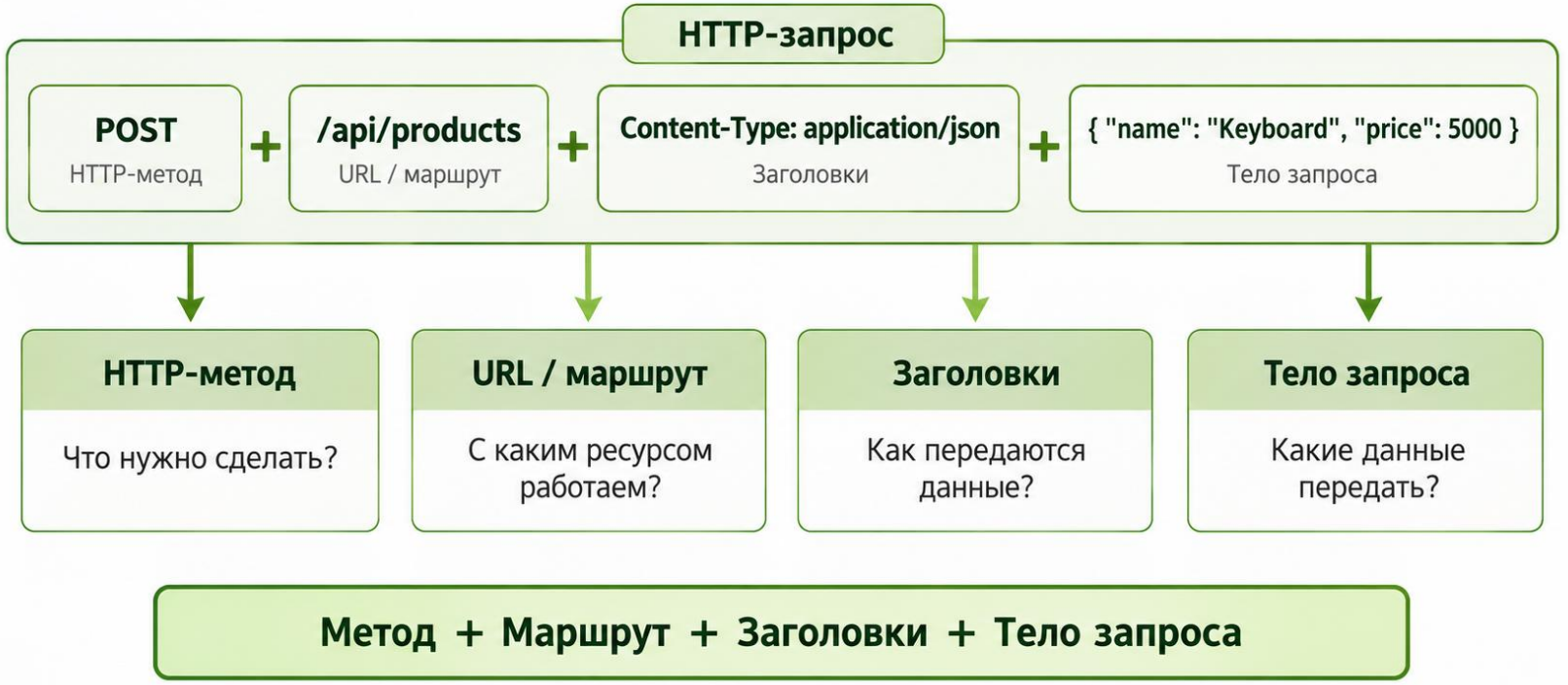
DELETE /api/products/5

Один и тот же адрес может обрабатываться по-разному в зависимости от HTTP-метода.
URL определяет ресурс, а HTTP-метод — действие над ним.

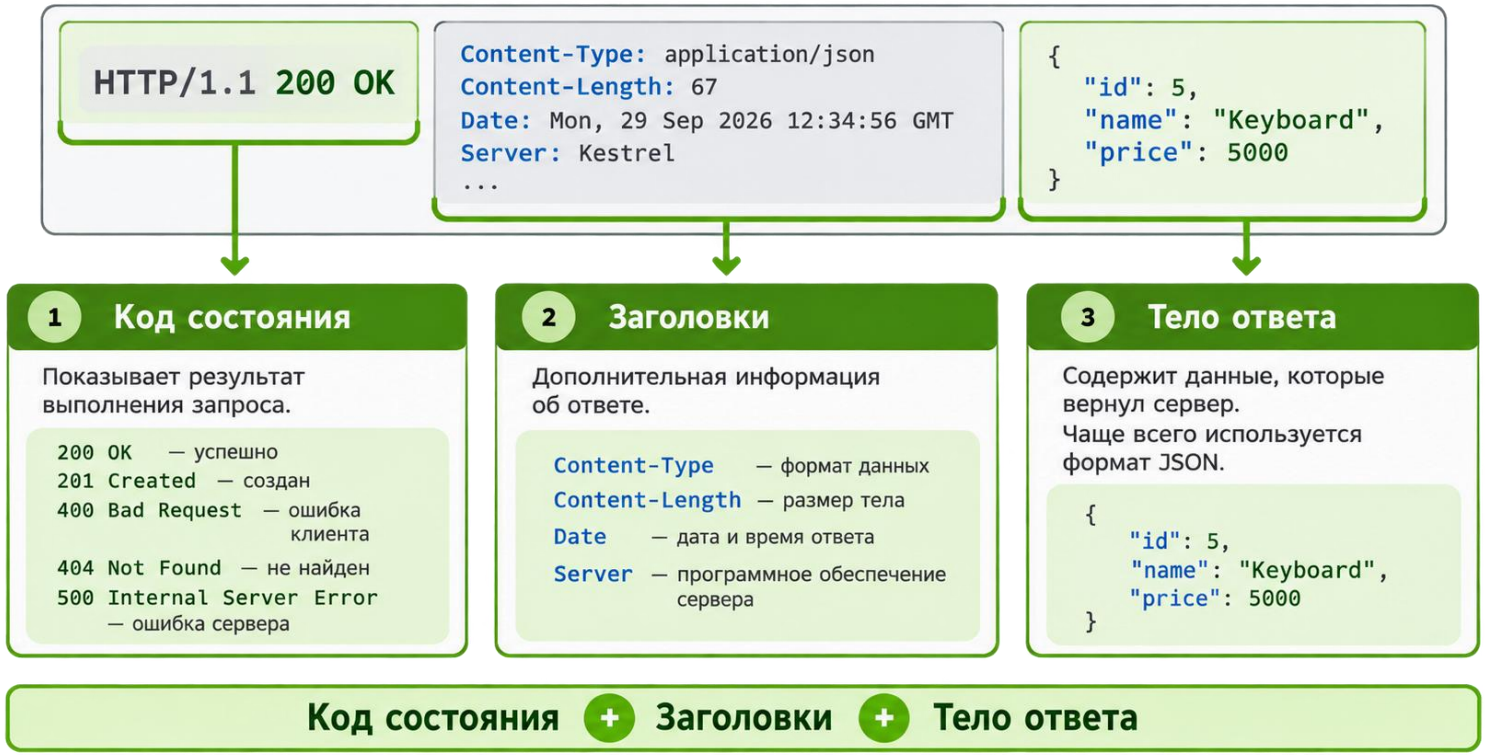
Коды состояния HTTP

Диапазон	Значение
1xx	Информационные ответы
2xx	Успешное выполнение
3xx	Перенаправление
4xx	Ошибка со стороны клиента
5xx	Ошибка на стороне сервера

Структура HTTP-запроса



Структура HTTP-ответа



Формат JSON

JSON (JavaScript Object Notation) — текстовый формат представления и передачи структурированных данных.

В Web API JSON часто используется как основной формат обмена между клиентом и сервером.

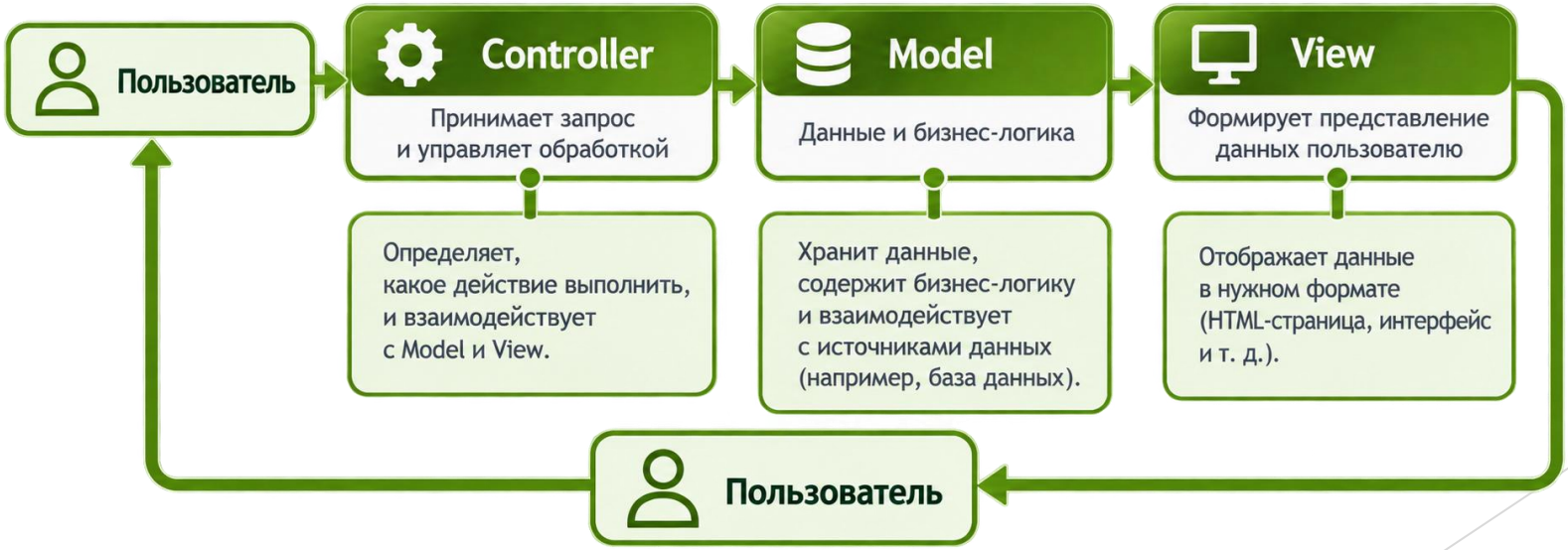

```
{
  "id": 5,
  "name": "Keyboard",
  "price": 5000,
  "inStock": true
}
```

Архитектурный паттерн MVC

MVC (Model-View-Controller) — архитектурный паттерн, который разделяет приложение на независимые части с разными обязанностями.

Три основных компонента:

- Model — данные и правила работы с ними;
- View — представление данных пользователю;
- Controller — принимает запрос и управляет обработкой.



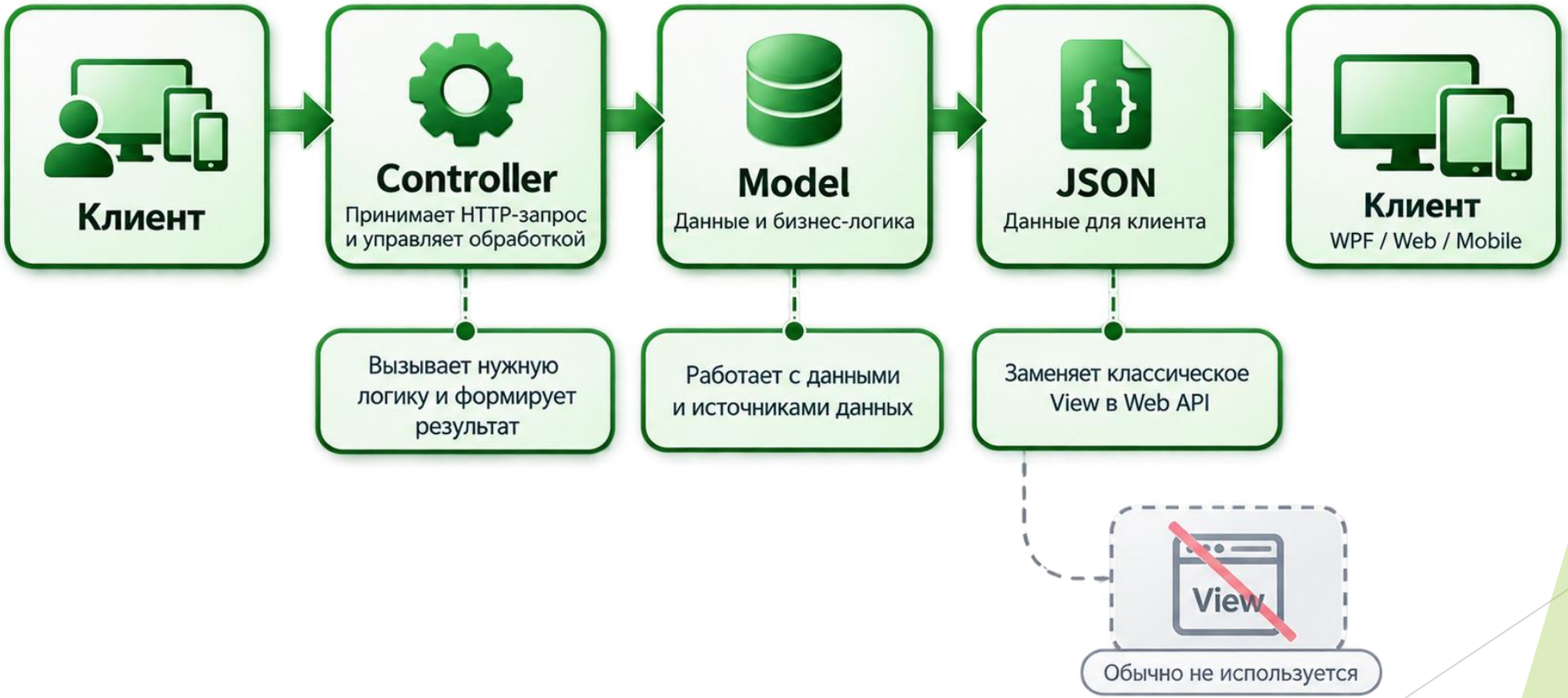
Зачем нужен MVC

MVC позволяет:

- разделить ответственность между частями приложения;
- уменьшить связанность кода;
- упростить сопровождение проекта;
- независимо изменять логику, данные и интерфейс;
- сделать структуру приложения более понятной.

MVC разделяет приложение по ответственности: данные, управление и представление.

MVC в Web API

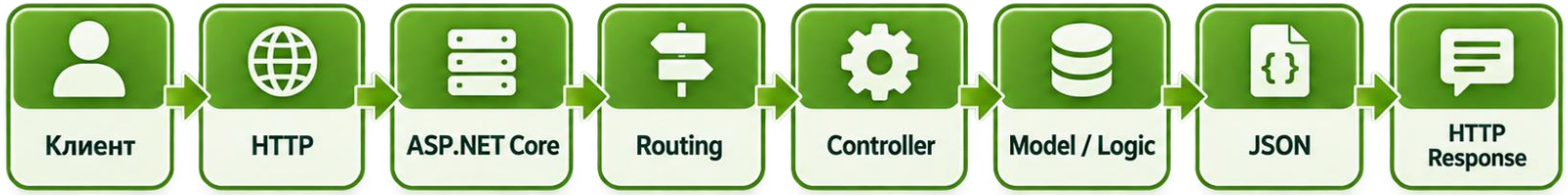

Клиент
Controller
Принимает HTTP-запрос и управляет обработкой
Model
Данные и бизнес-логика
JSON
Данные для клиента
Клиент
WPF / Web / Mobile
Вызывает нужную логику и формирует результат
Работает с данными и источниками данных
Заменяет классическое View в Web API
View
Обычно не используется

ASP.NET Core и Web API

ASP.NET Core — кроссплатформенный фреймворк .NET для создания веб-приложений, сервисов и Web API.

С его помощью можно разрабатывать:

- Web API;
- веб-приложения;
- серверные части мобильных и desktop-приложений;
- микросервисы;
- real-time приложения.

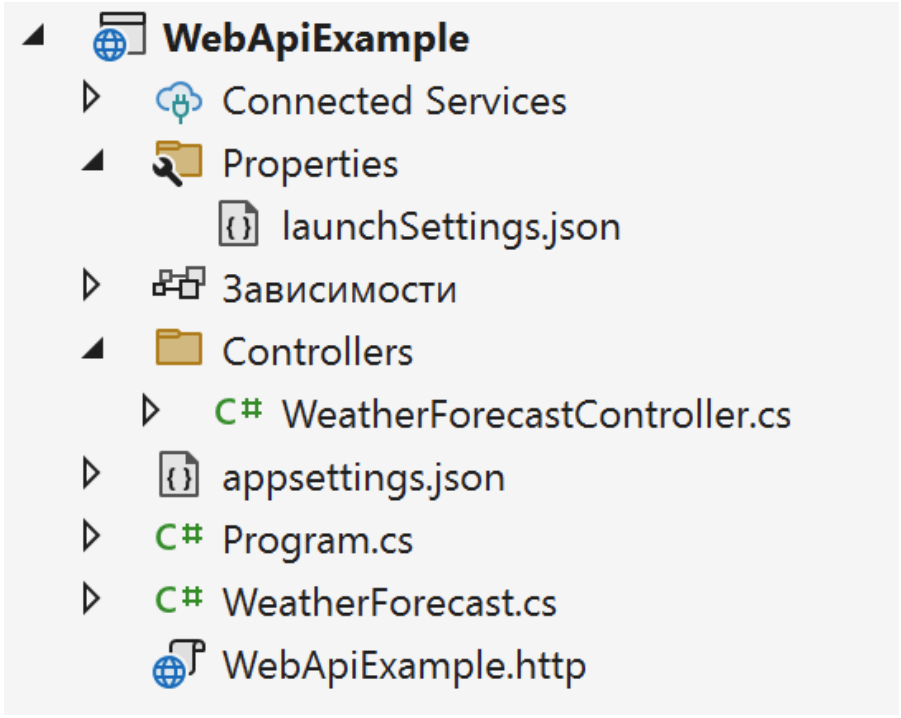


Преимущества ASP.NET Core

- работает на Windows, Linux и macOS;
- тесно интегрирован с .NET;
- имеет встроенную поддержку Web API;
- автоматически сериализует объекты в JSON;
- поддерживает контроллеры и маршрутизацию;
- хорошо подходит для создания REST API.

ASP.NET Core предоставляет готовую инфраструктуру для обработки HTTP-запросов и построения Web API.

Структура проекта ASP.NET Core Web API



Файл Program.cs

```
var builder = WebApplication.CreateBuilder(args);


builder.Services.AddControllers();

builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

var app = builder.Build();

// Configure the HTTP request pipeline.
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();

app.UseAuthorization();


app.MapControllers();

app.Run();
```

Контроллер в ASP.NET Core

```
[ApiController]
[Route("api/v1/[controller]")]
public class ProductsController : ControllerBase
{
    [HttpGet]
    public IActionResult GetAll()
    {
        return Ok();
    }

    [HttpGet("{id:int}")]
    public IActionResult Get(int id)
    {
        return Ok();
    }

    [HttpPost]
    public IActionResult Create(Product product)
    {
        return Ok();
    }
}
```

Action-методы контроллера

Action — это метод контроллера, который обрабатывает конкретный HTTP-запрос.

ASP.NET Core определяет нужный action по:

- HTTP-методу;
- маршруту;
- параметрам маршрута.

```
[HttpGet]
public IActionResult GetAll()
{
    return Ok();
}
```

Основные HTTP-атрибуты

- [HttpGet] – получение данных
- [HttpPost] – создание нового ресурса
- [HttpPut] – полное изменение существующего ресурса
- [HttpPatch] – частичное изменение существующего ресурса
- [HttpDelete] – удаление ресурса

Контроллер определяет общий маршрут, а action-методы — конкретные операции над ресурсом.

Параметры запроса и Model Binding

ASP.NET Core умеет автоматически получать данные из HTTP-запроса и передавать их в параметры action-метода.

Этот механизм называется Model Binding.

Основные источники данных:

- маршрут;
- query string;
- тело запроса;
- заголовки.

Пример

Model Binding связывает данные HTTP-запроса с параметрами и объектами C#.

```
[HttpGet]
public IActionResult GetAll(decimal? minPrice)
{
    return Ok();
}

[HttpGet("{id:int}")]
public IActionResult Get(int id)
{
    return Ok();
}

[HttpPost]
public IActionResult Create(Product product)
{
    return Ok();
}
```

Явное указание источника данных

ASP.NET Core часто сам определяет, откуда брать значения для параметров метода, но источник можно указать явно с помощью атрибутов.

Основные атрибуты:

- [FromRoute]
- [FromQuery]
- [FromBody]
- [FromHeader]

Пример

```
[HttpGet]
public IActionResult GetAll([FromQuery] decimal? minPrice)
{
    return Ok();
}

[HttpGet("{id:int}")]
public IActionResult Get([FromRoute] int id)
{
    return Ok();
}

[HttpPost]
public IActionResult Create([FromBody] Product product)
{
    return Ok();
}

public IActionResult Get([FromHeader(Name = "X-Client-Id")] string clientId)
{
    return Ok();
}
```

Формирование ответа IActionResult

Action-метод должен сообщить ASP.NET Core, какой HTTP-ответ вернуть клиенту.

Основные методы формирования ответа:

Метод	HTTP-код
`Ok()`	200
`Created()`	201
`NoContent()`	204
`BadRequest()`	400
`NotFound()`	404

Формирование ответа IActionResult

- IActionResult не сериализует данные сам. Он описывает результат, который ASP.NET Core должен превратить в HTTP-ответ.
- IActionResult позволяет гибко формировать HTTP-ответ в зависимости от результата выполнения action-метода.

Пример

```
[HttpGet("{id:int}")]
public IActionResult Get(int id)
{
    Product? product = FindProduct(id);

    if (product == null)
        return NotFound();

    return Ok(product);
}
```

Полный путь HTTP-запроса в ASP.NET Core

